TASK SCHEDULER USING BINARY HEAP

Abstract:

This project presents the design and implementation of a Priority-Based Task Scheduler developed in the C programming language. Traditional scheduling systems often rely on simple First-In-First-Out (FIFO) queues, which fail to prioritize critical tasks over routine operations. To address this limitation, this system utilizes a Binary Min-Heap data structure to manage tasks based on their urgency levels.

The core of the implementation lies in the efficiency of the heap algorithms, specifically Heapify-Up and Heapify-Down, which allow for task insertion and extraction in O(log n) time. This logarithmic complexity ensures the system remains scalable and responsive even as the volume of tasks increases. Additionally, the scheduler incorporates a Timestamp-based Tie-Breaking mechanism to ensure fairness and stability when multiple tasks share the same priority.

The final software provides a robust command-line interface for real-time task management, demonstrating how non-linear data structures can optimize resource allocation. The results indicate that the Binary Heap-based approach significantly outperforms linear search methods, making it a viable model for applications in operating systems, network traffic management, and real-time simulation environments.

Introduction:

Overview:

A Task Scheduler is a core component of modern computing environments, responsible for managing the execution of various processes, threads, or network packets. While a standard queue follows the First-In-First-Out (FIFO) principle, many real-world applications require certain tasks to be addressed before others, regardless of their arrival time. This project implements a Priority-Based Task Scheduler using a specialized data structure known as a Binary Heap.

The Need for Priority Scheduling:

In a standard system, a low-priority background task (like a system log update) might block a high-priority user request (like an emergency security patch) if it arrived first. Priority scheduling solves this by assigning a numerical “weight” to each task.

Key Applications Include:

Operating Systems: Managing CPU interrupts and process scheduling.

Networking: Prioritizing voice and video data over standard web traffic to prevent lag.

Bandwidth Management: Ensuring critical data packets are transmitted first during congestion.

Selection of the Binary Heap:To manage these priorities, we use a Binary Heap (specifically a Min-Heap). Unlike a simple sorted array, which requires O(n) time to insert a new element, a Binary Heap allows both insertion and the removal of the highest-priority task in Logarithmic Time (O(log n)).

Project Objectives:

- The primary objective of this mini-project is to design and develop a functional Priority-Based Task Scheduler that optimizes the management of processes based on their urgency. The specific technical and functional goals include:
- Implementation of a Non-Linear Data Structure: To move beyond basic linear structures and successfully implement a Binary Min-Heap in the C language.
- Efficiency Optimization: To ensure that the critical operations of task insertion and the extraction of the highest-priority task are performed in logarithmic time complexity, O(log n), to maintain system responsiveness.
- Data Integrity and Order: To develop a robust Task structure capable of storing multiple attributes (ID, Description, and Priority) while maintaining a tie-breaking mechanism via timestamps to handle tasks with identical priorities.
- Heap Governance: To master the internal logic of “Heapify-Up” and “Heapify-Down” algorithms, ensuring the heap property is preserved after every dynamic change to the queue.
- User Interaction: To provide a practical, menu-driven command-line interface (CLI) that allows for real-time interaction, testing, and visualization of the heap’s internal state.

Problem Statement:

Standard queues follow First-In-First-Out (FIFO). However, in real-world scenarios, some tasks (like security patches or system interrupts) require immediate attention. We need a data structure that:

Allows adding tasks with varying priority.

Always retrieves the task with the highest priority (lowest numerical value) in O(log n) time.

Data Structure and Algorithm:

The Choice of Data Structure: Binary Min-Heap

The project utilizes a Binary Min-Heap to implement the Priority Queue. A Binary Heap is a complete binary tree that satisfies the Heap Property: the value of the root node is the minimum (in a Min-Heap) of all values in the tree, and this relationship is recursively true for all subtrees.

Array-Based Representation:

To optimize memory and performance, the heap is stored in a contiguous array rather than using pointers. This allows for O(1) access to any node. For a task stored at index i:

Parent: parent(i) =(i-1)/2

Left Child: left(i) = 2i + 1

Right Child: right(i) = 2i + 2

Algorithmic Logic:

A. Insertion (Heapify-Up)

When a new task is added, it is initially placed at the next available leaf position (the end of the array) to maintain the "complete tree" property. However, this may violate the Min-Heap property.

Compare the new task's priority with its parent.

If the new task has a higher priority (smaller number), swap them.

Repeat this process until the task reaches the root or its parent has a higher priority.

Time Complexity: O(log n)

B. Extraction (Heapify-Down)

Removing the highest-priority task involves extracting the root node (index 0).

Replace the root with the last element in the array.

Decrease the heap size.

Compare the new root with its children.

Swap with the child that has the higher priority (smallest value).

Repeat this "sinking" process until the heap property is restored.

Time Complexity: O(\log n)

C. The Comparison Algorithm (Tie-Breaking)

To ensure the scheduler is “fair,” we implemented a custom comparison logic. If two tasks have the same priority value, the algorithm checks the timestamp. The task that arrived earlier is given precedence, effectively applying a FIFO (First-In-First-Out) rule within the same priority level.

Implementation Flow

The algorithm follows a structured lifecycle:

Initialize: Create an empty array and set size = 0.

Enqueue: Accept task details, assign a timestamp, and perform heapifyUp.

Dequeue: Extract the task at index 0, move the last element to the root, and perform heapifyDown.

Display: Traverse the array to list all pending tasks in their current heap order.

System Design:

Architecture Overview

The system is designed using a modular approach, separating the data storage from the management logic. It follows a Model-View-Controller (MVC) inspired pattern where:

The Model: Defines the Task and PriorityQueue structures.

The Controller: Contains the heap manipulation logic (heapifyUp, heapifyDown, push, pop).

The View: The command-line menu that interacts with the user.

Data Component Design

Each task is encapsulated in a struct, ensuring that all metadata travels with the priority value.

Task Structure: Contains id (int), description (char array), priority (int), and timestamp (long).

Priority Queue Structure: Contains an array of Task objects and a size counter to track the current number of active processes.

Functional Flow

The system operates through a structured decision-making process illustrated by the following logic:

Input Phase: The user provides task metadata. A unique timestamp is generated to ensure Priority-Timestamp Stability.

Processing Phase (The “Engine”): The Binary Heap logic determines the task’s position. This is the “black box” where the O(log n) efficiency occurs.

Execution Phase: The scheduler always targets heap[0], ensuring that the CPU/resource is always allocated to the most urgent requirement.

Implementation:

Environment and Tools

The system was developed and tested using the following environment:

Language: C (C11 Standard)

Compiler: GCC (GNU Compiler Collection)

Operating System: Windows/Linux

Libraries: <stdio.h> for I/O, <stdlib.h> for memory/exit functions, <string.h> for string manipulation, and <time.h> for generating unique task timestamps.

```c
//TASK SCHEDULERS USING BINARY HEAP
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX 100

// Structure to define a Task
typedef struct {
    int id;
    char description[100];
    int priority;     // Lower number = Higher priority
    long timestamp;   // For tie-breaking (FCFS)
} Task;

// Structure for the Priority Queue (Min-Heap)
typedef struct {
    Task heap[MAX];
    int size;
} PriorityQueue;

// Function to swap two tasks
void swap(Task *a, Task *b) {
    Task temp = *a;
    *a = *b;
```

```c
}

// Comparison logic: Priority first, then Timestamp for ties
int compare(Task a, Task b) {
    if (a.priority < b.priority) return -1;
    if (a.priority > b.priority) return 1;
    if (a.timestamp < b.timestamp) return -1;
    return 1;
}

// Restore heap property from bottom to top
void heapifyUp(PriorityQueue *pq, int index) {
    int parent = (index - 1) / 2;
    while (index > 0 && compare(pq->heap[index], pq->heap[parent]) < 0) {
        swap(&pq->heap[index], &pq->heap[parent]);
        index = parent;
        parent = (index - 1) / 2;
    }
}

// Restore heap property from top to bottom
void heapifyDown(PriorityQueue *pq, int index) {
    int smallest = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

```

```c
    if (left < pq->size && compare(pq->heap[left], pq->heap[smallest]) < 0)
        smallest = left;
    if (right < pq->size && compare(pq->heap[right], pq->heap[smallest]) < 0)
        smallest = right;

    if (smallest != index) {
        swap(&pq->heap[index], &pq->heap[smallest]);
        heapifyDown(pq, smallest);
    }
}

// Add a new task to the scheduler
void addTask(PriorityQueue *pq) {
    if (pq->size >= MAX) {
        printf("\n[!] Error: Scheduler queue is full!\n");
        return;
    }

    Task t;
    printf("\n--- Create New Task ---\n");
    printf("Enter Task ID (Integer): ");
    scanf("%d", &t.id);
    printf("Enter Description: ");
    scanf(" %[^\n]s", t.description);
    printf("Enter Priority (1-10, where 1 is urgent): ");
    scanf("%d", &t.priority);
```

```c
    scanf("%d", &t.priority);

    // Using a counter-based timestamp to ensure unique arrival order
    t.timestamp = time(NULL) + pq->size;

    pq->heap[pq->size] = t;
    heapifyUp(pq, pq->size);
    pq->size++;

    printf("[+] Task '%s' added successfully.\n", t.description);
}

// Remove and execute the highest priority task
void executeTask(PriorityQueue *pq) {
    if (pq->size == 0) {
        printf("\n[!] No pending tasks to execute.\n");
        return;
    }

    Task top = pq->heap[0];
    pq->heap[0] = pq->heap[pq->size - 1];
    pq->size--;
    heapifyDown(pq, 0);

    printf("\n>>> EXECUTING: [ID: %d] %s (Priority: %d)\n", top.id, top
        .description, top.priority);
```

```c
}

// Display tasks in their current heap storage order
void displayQueue(PriorityQueue *pq) {
    if (pq->size == 0) {
        printf("\n[i] The queue is currently empty.\n");
        return;
    }

    printf("\n--- Current Task Queue (Heap Order) ---\n");
    printf("%-5s %-20s %-10s\n", "ID", "Description", "Priority");
    for (int i = 0; i < pq->size; i++) {
        printf("%-5d %-20s %-10d\n", pq->heap[i].id, pq->heap[i].description, pq
            ->heap[i].priority);
    }
}

int main() {
    PriorityQueue pq = { .size = 0 };
    int choice;

    while (1) {
        printf("\n==============================");
        printf("\n   TASK SCHEDULER SYSTEM");
        printf("\n==============================");
```

```c
        printf("\n1. Add New Task");
        printf("\n2. Execute Next Priority Task");
        printf("\n3. View All Pending Tasks");
        printf("\n4. Exit");
        printf("\nSelect Option: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Exiting.\n");
            break;
        }

        switch (choice) {
            case 1: addTask(&pq); break;
            case 2: executeTask(&pq); break;
            case 3: displayQueue(&pq); break;
            case 4:
                printf("Shutting down scheduler. Goodbye!\n");
                exit(0);
            default:
                printf("[!] Invalid selection. Please try again.\n");
        }
    }
    return 0;
}
```

Output:

```
==============================
   TASK SCHEDULER SYSTEM
==============================
1. Add New Task
2. Execute Next Priority Task
3. View All Pending Tasks
4. Exit
Select Option: 1

--- Create New Task ---
Enter Task ID (Integer): 101
Enter Description: quizz
Enter Priority (1-10, where 1 is urgent): 3
[+] Task 'quizz' added successfully.

==============================
   TASK SCHEDULER SYSTEM
==============================
1. Add New Task
2. Execute Next Priority Task
3. View All Pending Tasks
4. Exit
Select Option: 1
```

```
--- Create New Task ---
Enter Task ID (Integer): 104
Enter Description: lab report
Enter Priority (1-10, where 1 is urgent): 1
[+] Task 'lab report' added successfully.

==============================
   TASK SCHEDULER SYSTEM
==============================
1. Add New Task
2. Execute Next Priority Task
3. View All Pending Tasks
4. Exit
Select Option: 3

--- Current Task Queue (Heap Order) ---
ID    Description          Priority
104   lab report           1
101   quizz                3

==============================
   TASK SCHEDULER SYSTEM
==============================
1. Add New Task
2. Execute Next Priority Task
```

```
3. View All Pending Tasks
4. Exit
Select Option: 1

--- Create New Task ---
Enter Task ID (Integer):
104
Enter Description: mini project
Enter Priority (1-10, where 1 is urgent): 2
[+] Task 'mini project ' added successfully.

==============================
   TASK SCHEDULER SYSTEM
==============================
1. Add New Task
2. Execute Next Priority Task
3. View All Pending Tasks
4. Exit
Select Option: 3

--- Current Task Queue (Heap Order) ---
ID    Description          Priority
104   lab report           1
101   quizz                3
104   mini project         2
```

```
==============================
   TASK SCHEDULER SYSTEM
==============================
1. Add New Task
2. Execute Next Priority Task
3. View All Pending Tasks
4. Exit
Select Option: 2

>>> EXECUTING: [ID: 104] lab report (Priority: 1)

==============================
   TASK SCHEDULER SYSTEM
==============================
1. Add New Task
2. Execute Next Priority Task
3. View All Pending Tasks
4. Exit
Select Option: 3

--- Current Task Queue (Heap Order) ---
ID    Description          Priority
104   mini project         2
101   quizz                3

==============================
```

```
==============================
   TASK SCHEDULER SYSTEM
==============================
1. Add New Task
2. Execute Next Priority Task
3. View All Pending Tasks
4. Exit
Select Option: 2

>>> EXECUTING: [ID: 104] mini project  (Priority: 2)

==============================
   TASK SCHEDULER SYSTEM
==============================
1. Add New Task
2. Execute Next Priority Task
3. View All Pending Tasks
4. Exit
Select Option: 3

--- Current Task Queue (Heap Order) ---
ID    Description          Priority
101   quizz                3
```

```
==============================
   TASK SCHEDULER SYSTEM
==============================
1. Add New Task
2. Execute Next Priority Task
3. View All Pending Tasks
4. Exit
Select Option: 4
Shutting down scheduler. Goodbye!
```

Key Code Modules

The implementation is divided into four critical functional segments:

A. Data Modeling

We use a struct to define a Task. This allows the scheduler to treat each process as a single object containing its ID, description, and priority level. The PriorityQueue struct then manages an array of these tasks.

B. The Comparison Logic

The compare() function acts as the “decision-making” engine. It returns a value based on priority; however, if two tasks have the same priority, it compares their timestamp. This ensures the scheduler remains stable and fair.

C. Maintenance Functions (Heapify)

heapifyUp: This is invoked during addTask. It ensures that a new, high-priority task “bubbles up” to its correct position in O(log n) time.

heapifyDown: This is invoked during executeTask. After the root is removed, it ensures the new root “sinks” to the correct level, maintaining the Min-Heap property.

Implementation Steps

Queue Initialization: The PriorityQueue is initialized with a size of 0.

Task Capture: The addTask function prompts the user for task details. It automatically attaches a time_t value to the task.

Positioning: The task is placed at heap[size] and heapifyUp is called to reorganize the tree.

Execution: The executeTask function always accesses heap[0]. It then replaces the root with the last element and calls heapifyDown.

Error Handling

Overflow: The addTask function checks if size >= MAX to prevent memory corruption.

Underflow: The executeTask function checks if size == 0 to prevent accessing null or garbage data when the qu0eue is empty.

Complexity Analysis:

Time Complexity

The efficiency of the Task Scheduler is determined by the height of the Binary Heap, which is log2 n for a tree with n nodes.

Operation	Complexity	Description
Insertion(addTask)	O(log n)	In the worst case, a new task with the highest priority must "bubble up" from the leaf to the root, traversing the height of the tree.
Extraction(Execute Task)	O(log n)	When the root is removed, the last element is moved to the top and must "heapify down" to its correct position, traversing the height of the tree.
Peek(view next)	O(1)	The highest-priority task is always located at index O of the array, allowing for instant access.
Heapify process	O(log n)	Both heapifyUp and heapifyDown are bounded by the height of the tree.

Space Complexity

The space complexity of the system is O(n), where n is the maximum number of tasks the scheduler can hold.

Static Allocation: In this implementation, we use a fixed-size array (MAX_SIZE). The memory is allocated at compile-time, ensuring predictable memory usage but limiting the total number of tasks to the predefined constant.

Auxiliary Space: The auxiliary space used during heap operations is O(1) for iterative heapifyUp and O(log n) for recursive heapifyDown due to the function call stack.

Advantages of the Binary Heap Scheduler:

High Efficiency (O(log n) Performance):

The primary advantage is the logarithmic time complexity. In a system with 1,000 tasks, a linear search (unsorted list) might take 1,000 comparisons to find the most urgent task. A Binary Heap performs the same operation in roughly 10 comparisons (log2 1000 ~ 10). This ensures the scheduler remains fast even as the workload grows.

Dynamic Ordering:

Unlike a sorted array, which is “static” and expensive to update, a Binary Heap is designed for frequent changes. Tasks can be added or removed at any time, and the structure

automatically re-balances itself. This makes it ideal for real-time systems like an operating system's CPU scheduler.

Minimal Memory Overhead:

By using an array-based representation, the system avoids the memory overhead associated with pointers in a linked list or a traditional tree. There are no "next" or "previous" pointers; the relationship between parent and child is calculated mathematically, saving significant RAM.

Guaranteed Urgency (Min-Heap Property):

The "Min-Heap" property ensures that the root is always the most urgent task. The system never has to scan the entire dataset to decide what to do next; it simply looks at the first element, providing O(1) access to the highest priority task.

Fair Tie-Breaking:

By integrating a timestamp, the implementation gains Stability. It prevents "Priority Collisions" where two equally important tasks might otherwise be handled in an unpredictable order. It combines the best of both worlds: Priority Scheduling and First-Come-First-Served (FCFS).

Disadvantages and Limitations:

Lack of Search Efficiency:

While a Binary Heap is excellent for accessing the top element, it is inefficient for searching for a specific task (e.g., finding Task ID 502).

The Issue: Because the heap is only partially ordered (parents are smaller than children, but left/right siblings have no set relationship), finding an arbitrary task requires a linear scan.

Complexity: O(n) compared to O(log n) in a Binary Search Tree (BST).

Fixed Memory Constraints:

In this specific implementation, we used a static array.

The Issue: If the number of tasks exceeds the predefined MAX_SIZE, the system will fail to add new tasks (Overflow). Conversely, if the array is large but only contains a few tasks, memory is wasted.

Workaround: This could be solved with dynamic memory allocation (realloc), but that adds complexity to the code logic.

Complexity of Modification

Updating a task that is already in the queue is mathematically expensive.

The Issue: If a user wants to change the priority of a task that is currently sitting in the middle of the heap, the system must perform a search (O(n)) to find it, update the value, and then perform both a heapifyUp and a heapifyDown to ensure the structure remains valid.

Stability Issues (Native Heap):

By default, a Binary Heap is not a stable data structure.

The Issue: Without the manual addition of a timestamp (which we included in this project), two tasks with the same priority might be swapped during the heapification process. In a raw heap, "first-come, first-served" is not guaranteed unless specifically coded.

Not Ideal for Broad Data Retrieval:

The heap is designed for one thing: getting the minimum.

The Issue: If the user wants to see a list of all tasks sorted perfectly from 1 to 100, the heap is not the best choice. Printing the heap array directly shows the "heap order," not a perfectly sorted list. To get a sorted list, you would have to perform "Heap Sort," which effectively empties the queue.

Applications:

The Binary Heap-based Priority Queue is a fundamental component in systems where task order depends on urgency rather than arrival time.

- Operating System Scheduling: Used to manage the "Ready Queue" where the CPU prioritizes critical system interrupts and user-interface tasks over background processes to ensure system responsiveness.
- Network Traffic Control: Routers prioritize time-sensitive data, such as Voice-over-IP (VoIP) and video streaming, over standard web traffic to maintain high Quality of Service (QoS) and reduce lag.
- Navigation Algorithms: Essential for Dijkstra's Shortest Path algorithm, used by GPS services to quickly extract the nearest unvisited location and calculate the fastest route in O(log n) time.
- Event-Driven Simulations: Used in scientific modeling to manage a timeline of events, ensuring that the event with the earliest timestamp is always processed next to maintain chronological accuracy.
- Hospital Triage Systems: Digital healthcare platforms use these queues to flag critical patients for immediate medical attention based on severity rather than their order of arrival in the emergency room.

Conclusion:

The implementation of the Priority-Based Task Scheduler successfully demonstrates the practical application of non-linear data structures in solving real-world computational challenges. By shifting from a traditional FIFO model to a Binary Min-Heap architecture, this project provides an efficient solution for environments where task urgency varies dynamically.

The core achievement of this project is the optimization of system performance. While simpler structures like unsorted or sorted arrays suffer from linear time complexities (O(n)) in either insertion or deletion, the Binary Heap ensures that the scheduler remains responsive with a logarithmic time complexity (O(log n)). This ensures that even as the volume of tasks increases, the system's latency remains minimal.

Furthermore, the inclusion of a timestamp-based tie-breaking mechanism addresses the inherent instability of standard heaps, ensuring that the scheduler is not only fast but also fair. The final C implementation provides a robust, menu-driven interface that effectively handles task creation, prioritization, and execution.

In summary, this mini-project highlights how informed data structure selection—specifically the Binary Heap—can drastically improve the efficiency of resource management systems, providing a foundation for more complex scheduling algorithms used in modern operating systems and high-traffic network routers.

Feature Enhancement:

While the current implementation of the Task Scheduler is efficient and functional, several enhancements can be integrated to handle more complex, real-world scenarios:

- Task Aging (Preemption Prevention):

In a standard priority queue, a "starvation" problem can occur where low-priority tasks never get executed because high-priority tasks are constantly being added.

Enhancement: Implement an Aging Algorithm that periodically increases the priority of tasks that have waited in the queue for a specific duration. This ensures that every task eventually reaches the root for execution.

- Dynamic Memory Management

The current version uses a static array with a fixed MAX_SIZE.

Enhancement: Use Dynamic Arrays (malloc and realloc). This would allow the scheduler to scale infinitely (limited only by system RAM), automatically doubling its capacity when the heap becomes full.

- Multi-Level Feedback Queues (MLFQ)

To mimic modern Operating Systems like Linux or Windows:

Enhancement: Transition from a single heap to Multiple Queues. Different heaps could handle different categories of tasks (e.g., System, User, and Interactive), each with different scheduling policies.

- Persistence (File I/O)

Currently, all tasks are lost once the program is closed.

Enhancement: Add File Handling capabilities to save the state of the heap to a .dat or .txt file on exit and reload the pending tasks upon startup.

- Graphical User Interface (GUI)

Enhancement: Transition the project from a Command Line Interface (CLI) to a GUI using libraries like GTK or Qt. This would allow for a visual representation of the Binary Heap tree structure, making it a valuable educational tool for visualizing data structure changes in real-time.

References:

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
2. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
3. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
4. Sahni, S. (2004). Data Structures, Algorithms, and Applications in C++. Silicon Press.
5. Knuth, D. E. (1998). The Art of Computer Programming, Volume 3: Sorting and Searching. Addison-Wesley.
6. Weiss, M. A. (2012). Data Structures and Algorithm Analysis in C. Pearson Education.
7. Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson.
8. Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
9. Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983). Data Structures and Algorithms. Addison-Wesley.
10. IEEE Computer Society. C11 Standard (ISO/IEC 9899:2011).